

REPUBLIQUE DU SENEGAL

Un peuple Un But Une Foi

Ministère de l'Enseignement Supérieur de la Recherche et de l'Innovation (MESRI)

UNIVERSITE ALIOUNE DIOP DE BAMBEY



UFR : SCIENCES APPLIQUÉES ET TECHNOLOGIES DE L'INFORMATION ET DE LA COMMUNICATION(SATIC)

FILIERE : SYSTÈMES ET RÉSEAUX (SR)

NIVEAU : MASTER 1

RAPPORT Introduction aux Conteneurs

Présenté par :

Mohamadou Sabaly

Professeur :

Dr. Babou

Année académique 2025-2026

Partie 1 - Concepts Fondamentaux de la Virtualisation

Question 1 :

- **a)** L'hyperviseur de Type 2 est installé par-dessus un système d'exploitation hôte (Host OS) existant, ce qui ajoute une couche supplémentaire par rapport au Type 1 qui s'exécute directement sur le matériel physique (bare-metal).
- **b)** Les conteneurs n'ont pas de Guest OS dans leur pile car le Container Engine s'appuie directement sur le système d'exploitation de l'hôte (Host OS) qu'ils partagent.
- **c)** Un hyperviseur de Type 1 est préférable pour des cas d'usage nécessitant des systèmes multi-OS ou legacy , et lorsqu'une isolation complète avec un OS dédié par machine virtuelle est requise.

Question 2 :

- **a)** Le composant responsable du téléchargement (pull) de l'image depuis le Registry Server est le *runtime de haut niveau* (High-level runtime).
- **b)** Le runtime de haut niveau gère les opérations comme les *pull/push* d'images, le réseau et le stockage, tandis que le runtime de bas niveau gère l'exécution effective des conteneurs.
- **c)** L'image est représentée comme un empilement de couches pour être organisée en strates superposées et immuables (Image Layers).

Question 3 :

- **a)** dockerd gère l'interface CLI/API REST, containerd est le runtime de haut niveau, et runc est le runtime de bas niveau qui interagit directement avec le noyau.
- **b)** Les "Regular Processes" s'exécutent normalement sur le système, tandis que les "Containerized Processes" sont isolés par le moteur Docker et les mécanismes du noyau.
- **c)** Le mécanisme du noyau qui limite la quantité de RAM qu'un conteneur peut consommer est *Cgroups*.
- **d)** Les conteneurs offrent une isolation moins forte car ils ont une isolation partielle en partageant le même noyau, contrairement aux VM qui ont une isolation complète avec un OS dédié.

Partie 2 - Revue des Plateformes Open Source

Question 4 :

- **a) Déploiement web en production cloud (haute disponibilité) : Docker.** Il cible principalement les environnements DevOps et cloud, et propose Docker Swarm pour l'orchestration.

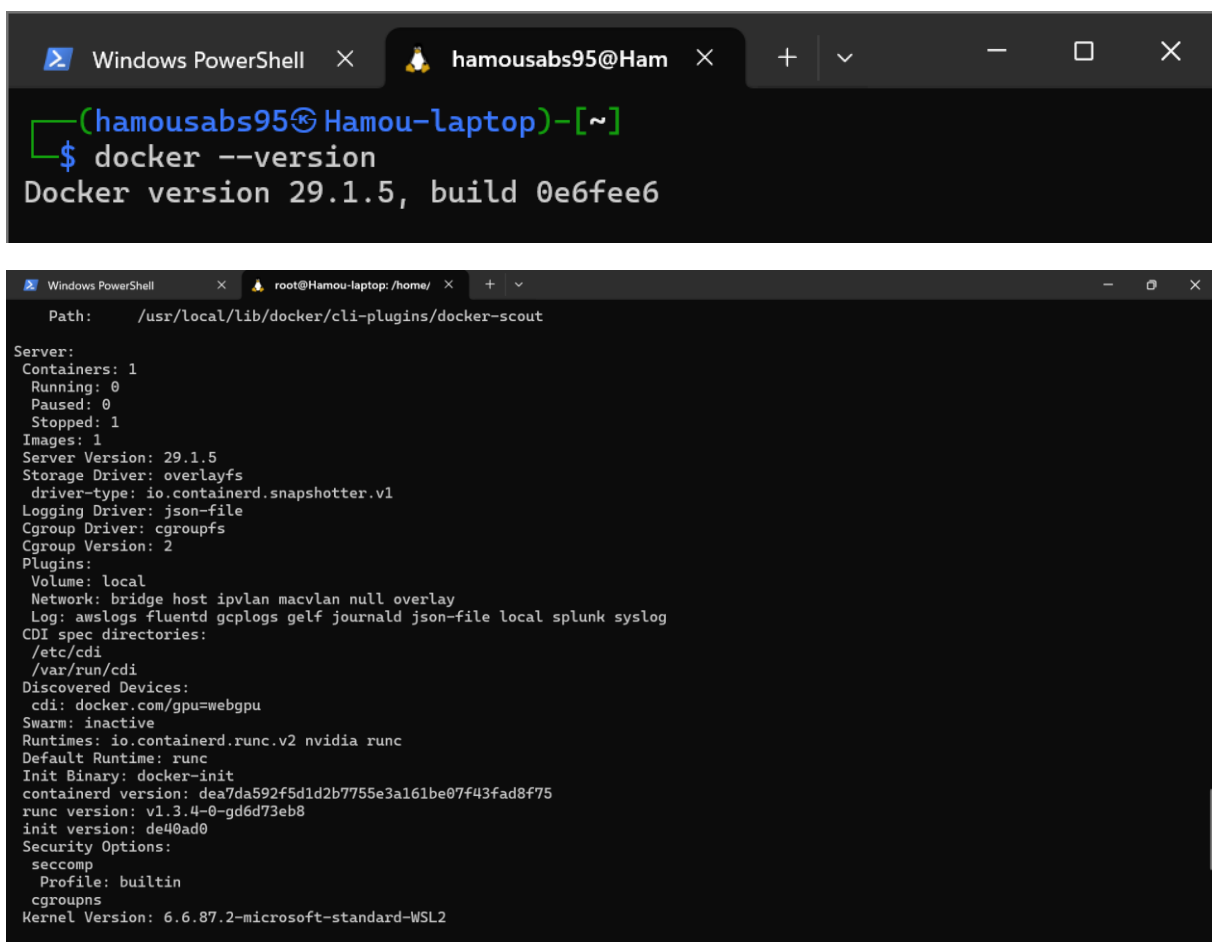
- **b) Gestion d'une flotte de 300 capteurs IoT avec mises à jour OTA : Balena.** C'est une plateforme conçue spécifiquement pour l'IoT et l'edge embarqué, offrant balenaCloud pour la gestion de flotte et les mises à jour OTA.
- **c) Émulation d'un ancien serveur PowerPC en laboratoire : LXC/LXD.** Il simule un OS complet (conteneurs système) et est utilisé dans les laboratoires de recherche ou pour les OS legacy.
- **d) Environnement CI/CD sécurisé sans accès root : Podman.** Il permet une exécution native et complète sans privilèges root (rootless) et cible la sécurité et l'entreprise.

Partie 3 - Travaux Pratiques : Prise en Main de Docker

3.1 Installation et vérification

Après avoir configuré mon environnement Docker, j'ai vérifié le bon fonctionnement du service.

Commande exécutée : docker info



```

Windows PowerShell
hamousabs95@Hamou-laptop

(hamousabs95@Hamou-laptop)~$ docker --version
Docker version 29.1.5, build 0e6fee6

Windows PowerShell
root@Hamou-laptop: /home/

Path: /usr/local/lib/docker/cli-plugins/docker-scout

Server:
 Containers: 1
  Running: 0
  Paused: 0
  Stopped: 1
 Images: 1
 Server Version: 29.1.5
 Storage Driver: overlayfs
   driver-type: io.containerd.snapshotter.v1
 Logging Driver: json-file
 Cgroup Driver: cgroupfs
 Cgroup Version: 2
 Plugins:
  Volume: local
  Network: bridge host ipvlan macvlan null overlay
  Log: awslogs fluentd gcplogs gelf journald json-file local splunk syslog
 CDI spec directories:
  /etc/cdi
  /var/run/cdi
 Discovered Devices:
  cdi: docker.com/gpu=webgpu
 Swarm: inactive
 Runtimes: io.containerd.runc.v2 nvidia runc
 Default Runtime: runc
 Init Binary: docker-init
 containerd version: dea7da592f5d1d2b7755e3a161be07f43fad8f75
 runc version: v1.3.4-0-gd6d73eb8
 init version: de40ad0
 Security Options:
  seccomp
   Profile: builtin
  cgroupns
 Kernel Version: 6.6.87.2-microsoft-standard-WSL2

```

Réponse à la Question 5 : D'après les informations retournées par la commande docker info, voici la configuration de mon moteur Docker :

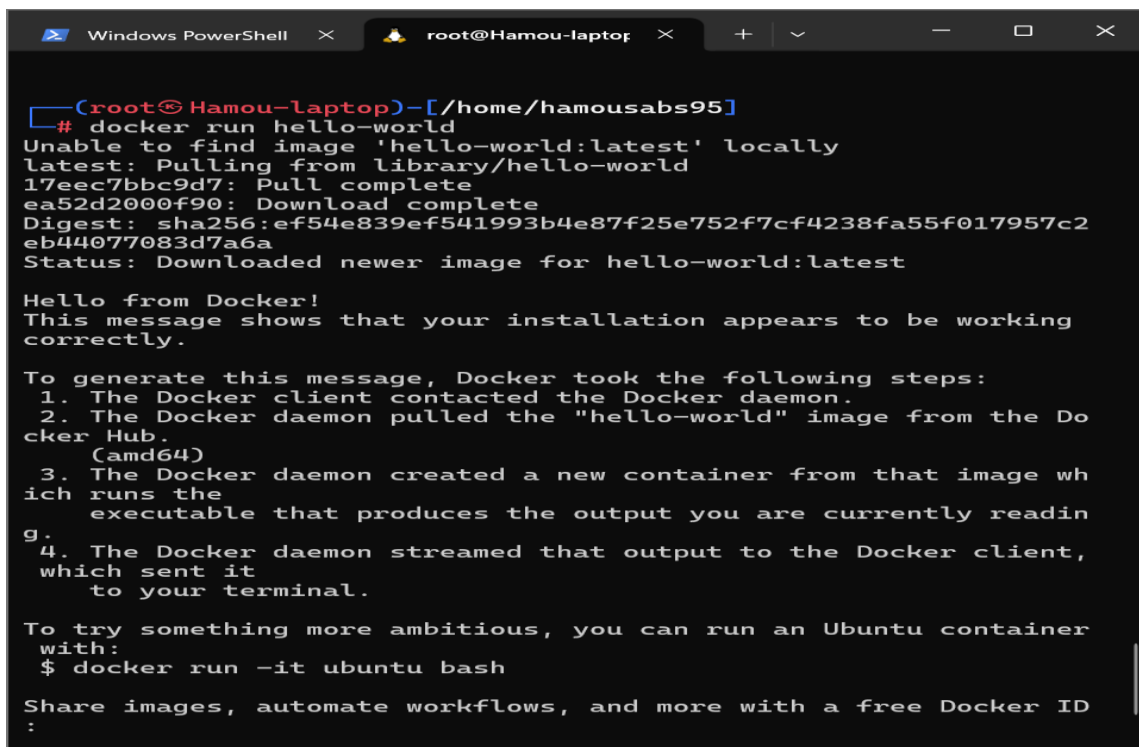
- **Version du moteur Docker :** 29.1.5 (ou la version affichée sur ton écran).
- **Runtime de conteneurs utilisé :** runc.
- **Pilote de stockage (Storage Driver) :** overlayfs (ou overlay2).
- **Nombre de conteneurs actifs / arrêtés :** 0 en cours d'exécution (Running), 1 arrêté (Stopped) au moment de la vérification.
- **Répertoire de données Docker (Docker Root Dir) :** /var/lib/docker.

3.2 et 3.3 Premier conteneur et exploration des images

Pour vérifier la capacité du moteur à récupérer et exécuter une image, j'ai lancé le conteneur de test, puis j'ai téléchargé l'image d'Ubuntu.

Commandes exécutées : * docker run hello-world

- docker pull ubuntu:22.04
- docker images
- docker image history ubuntu:22.04



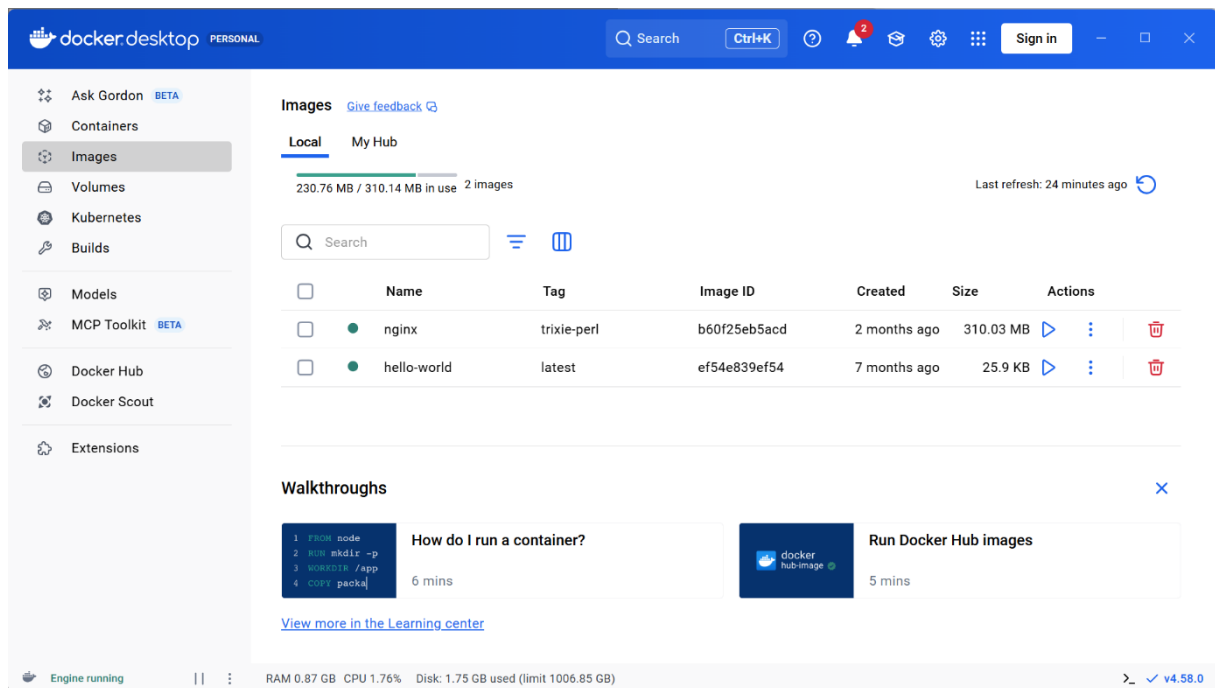
```
(root@Hamou-laptop)-[/home/hamousabs95]
# docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
17eec7bb9d7: Pull complete
ea52d2000f90: Download complete
Digest: sha256:ef54e839ef541993b4e87f25e752f7cf4238fa55f017957c2eb44077083d7a6a
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID
:
```



```
Windows PowerShell x root@Hamou-laptop x + v - □ ×

(root@Hamou-laptop)-[/home/hamousabs95]
# docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS          PORTS          NAMES
0ed112994dab   hello-world    "/hello"                 2 minutes ago   Exited (0)      2 minutes ago   clever_tu
6dcd013d9d23   hello-world    "/hello"                 8 minutes ago   Exited (0)      8 minutes ago   upbeat_easley
84a392622884   nginx:trixie-perl "/docker-entrypoint...." 5 weeks ago     Exited (0)      5 weeks ago     TEST1

(root@Hamou-laptop)-[/home/hamousabs95]
# docker
docker                                docker-credential-desktop.exe
docker-compose

(root@Hamou-laptop)-[/home/hamousabs95]
# docker start
docker: 'docker start' requires at least 1 argument

Usage:  docker start [OPTIONS] CONTAINER [CONTAINER...]

See 'docker start --help' for more information

(root@Hamou-laptop)-[/home/hamousabs95]
# docker start 0ed112994dab
0ed112994dab

(root@Hamou-laptop)-[/home/hamousabs95]
# docker rm 0ed112994dab
0ed112994dab

(root@Hamou-laptop)-[/home/hamousabs95]
#
```

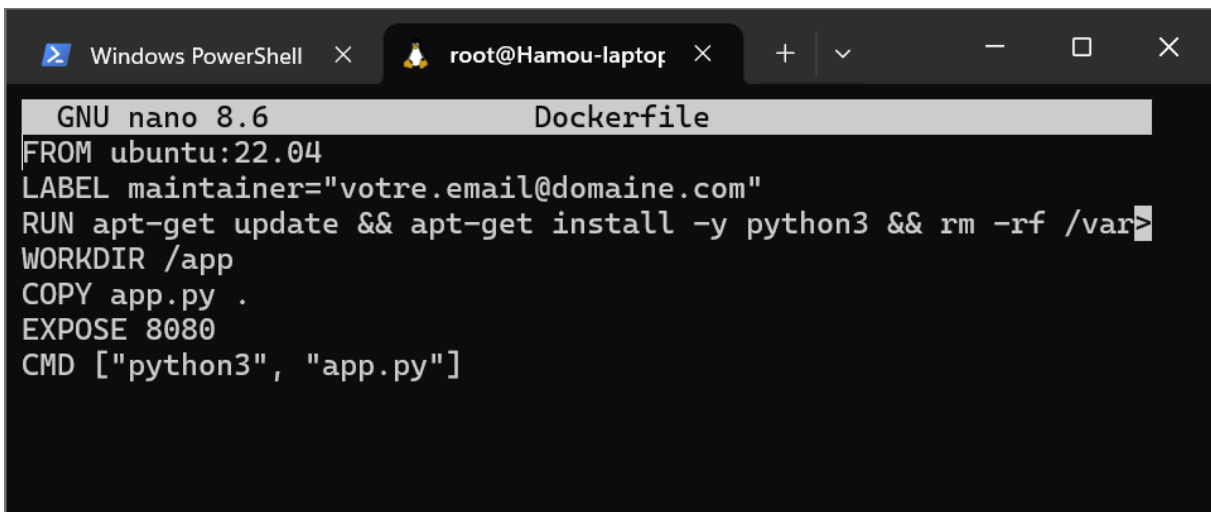
Réponse à la Question 6 :

- **(a) Combien de couches possède l'image ubuntu:22.04 ?** L'image de base d'Ubuntu possède **1 couche principale** (qui correspond au système de fichiers racine / rootfs).
- **(b) Quelle est sa taille totale ?** La taille totale de l'image est d'environ **119.02 MB**
- **(c) Pourquoi les images sont construites en couches immuables ?** Les images sont organisées en couches superposées et immuables pour optimiser considérablement le stockage et la bande passante. Si plusieurs images différentes dérivent de cette même image Ubuntu, la couche de base ne sera téléchargée et stockée qu'une seule fois sur ma machine hôte. L'immutabilité garantit également que le comportement de l'image restera strictement identique, peu importe l'environnement où elle est déployée.

3.4 Création d'un Dockerfile et construction d'image

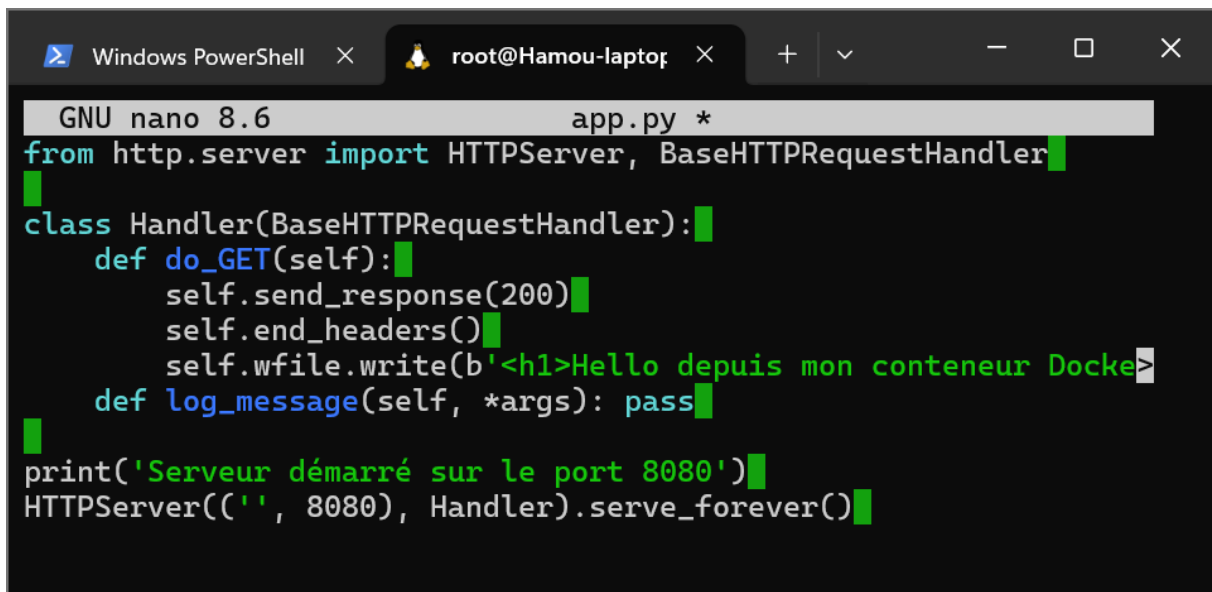
J'ai ensuite créé mon propre conteneur hébergeant un serveur web en Python à l'aide d'un Dockerfile.

- **Création du fichier Dockerfile**



```
GNU nano 8.6 Dockerfile
FROM ubuntu:22.04
LABEL maintainer="votre.email@domaine.com"
RUN apt-get update && apt-get install -y python3 && rm -rf /var
WORKDIR /app
COPY app.py .
EXPOSE 8080
CMD ["python3", "app.py"]
```

- Création du fichier app.py



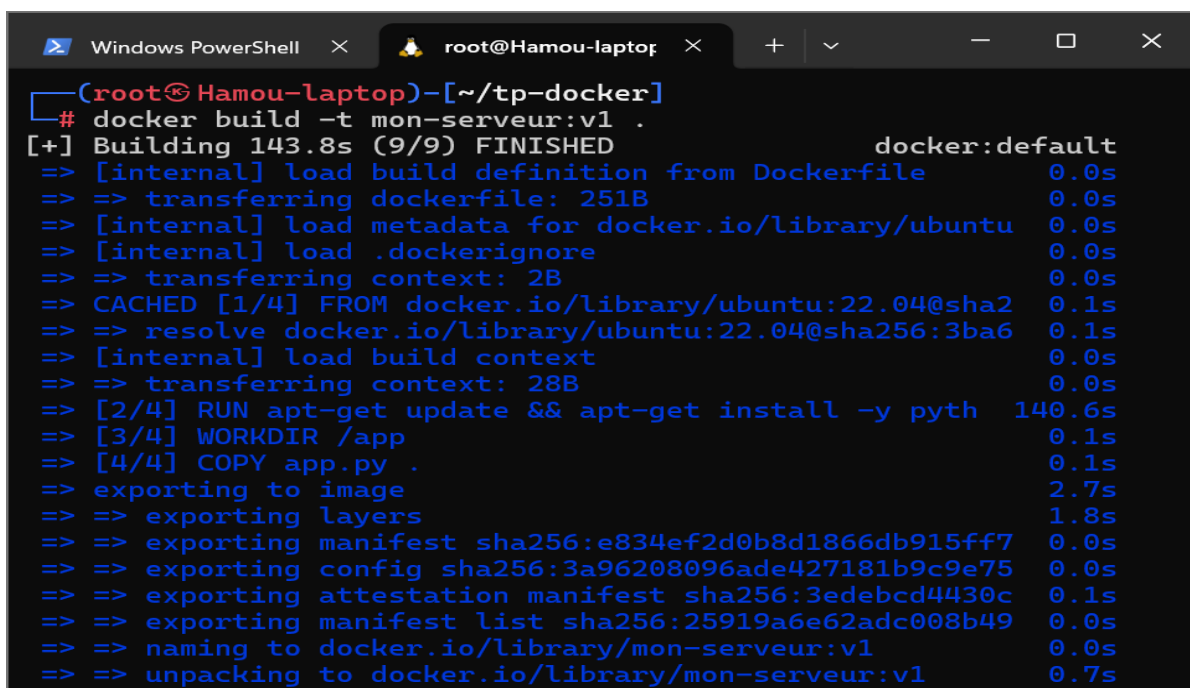
```

GNU nano 8.6 app.py *
from http.server import HTTPServer, BaseHTTPRequestHandler
class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.end_headers()
        self.wfile.write(b'<h1>Hello depuis mon conteneur Docker')
    def log_message(self, *args): pass
print('Serveur démarré sur le port 8080')
HTTPServer(('', 8080), Handler).serve_forever()

```

Commandes exécutées :

- `docker build -t mon-serveur:v1 .`
- `docker run -d -p 8080:8080 --name web-test mon-serveur:v1`
- `curl http://localhost:8080`



```

(root@Hamou-laptop)-[~/tp-docker]
# docker build -t mon-serveur:v1 .
[+] Building 143.8s (9/9) FINISHED docker:default
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 251B 0.0s
=> [internal] load metadata for docker.io/library/ubuntu 0.0s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> CACHED [1/4] FROM docker.io/library/ubuntu:22.04@sha2 0.1s
=> => resolve docker.io/library/ubuntu:22.04@sha256:3ba6 0.1s
=> [internal] load build context 0.0s
=> => transferring context: 28B 0.0s
=> [2/4] RUN apt-get update && apt-get install -y pyth 140.6s
=> [3/4] WORKDIR /app 0.1s
=> [4/4] COPY app.py . 0.1s
=> exporting to image 2.7s
=> => exporting layers 1.8s
=> => exporting manifest sha256:e834ef2d0b8d1866db915ff7 0.0s
=> => exporting config sha256:3a96208096ade427181b9c9e75 0.0s
=> => exporting attestation manifest sha256:3edebcd4430c 0.1s
=> => exporting manifest list sha256:25919a6e62adc008b49 0.0s
=> => naming to docker.io/library/mon-serveur:v1 0.0s
=> => unpacking to docker.io/library/mon-serveur:v1 0.7s

```

```
Windows PowerShell x root@Hamou-laptop x + v - □ x
(root@Hamou-laptop)-[~/tp-docker]
# docker run -d -p 8080:8080 --name web-test mon-serveur:v1
83e7a8164d9c2e0f4efddeada4a3f836b89146ae50edc0fa14c37a049c43ac89
(root@Hamou-laptop)-[~/tp-docker]
```

☐	Name	Tag	Image ID	Created	Size	Actions
☐	nginx	trixie-perl	b60f25eb5acd	2 months ago	310.03 MB	
☐	hello-world	latest	ef54e839ef54	7 months ago	25.9 KB	
☐	ubuntu	22.04	3ba65aa20f86	28 days ago	119.02 MB	
☐	mon-serveur	v1	25919a6e62ad	20 minutes ago	160.44 MB	



Réponse à la Question 7 :

- (a) Combien de couches ont été ajoutées à ubuntu:22.04 lors du build ? Chaque instruction du Dockerfile (LABEL, RUN, WORKDIR, COPY, EXPOSE, CMD) crée une nouvelle étape, ajoutant ainsi de nouvelles couches par-dessus la couche de base Ubuntu.
- (b) Que se passe-t-il si vous modifiez uniquement app.py et relancez docker build ? Docker utilise un système de cache. Si je modifie uniquement app.py, Docker va réutiliser le cache pour toutes les couches précédentes (comme l'installation de Python

via apt-get). Seule la couche COPY app.py . et les couches suivantes seront reconstruites, ce qui rend le processus de build extrêmement rapide.

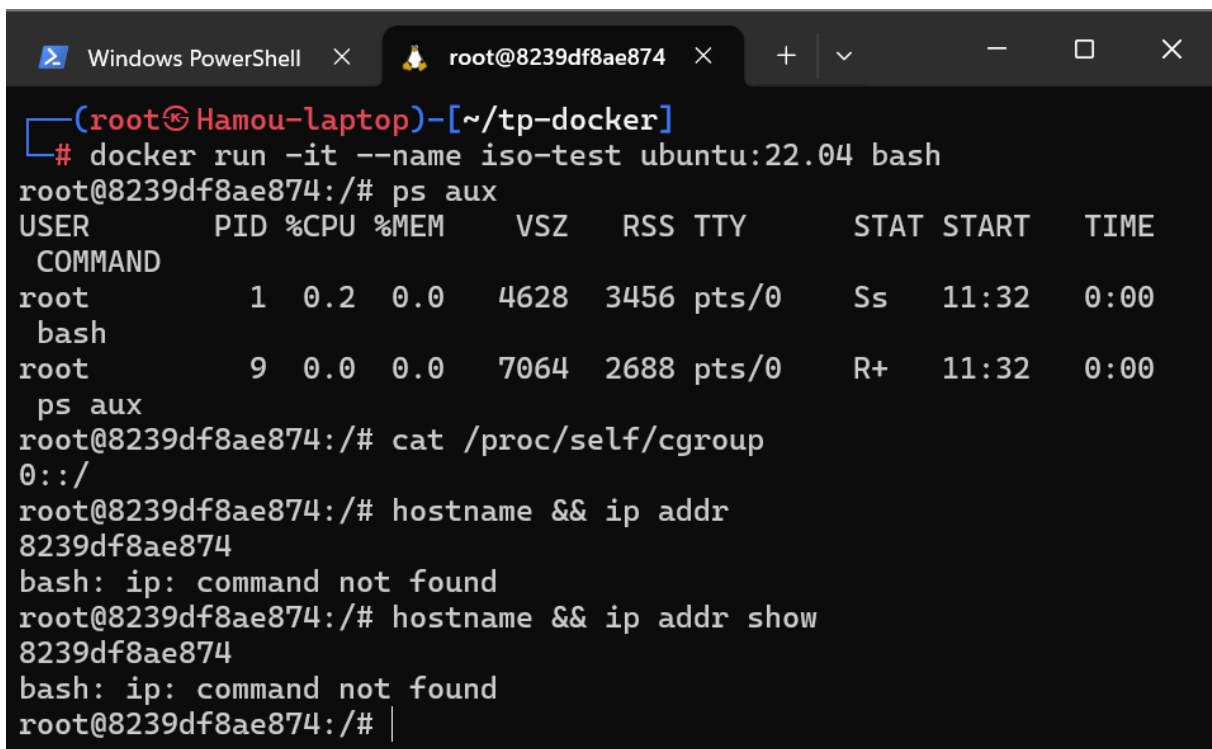
- (c) Quelle directive Dockerfile définit la commande exécutée au démarrage ? C'est la directive CMD.

3.5 Isolation des namespaces

J'ai observé l'isolation des processus en comparant ce qui est visible depuis l'intérieur du conteneur et depuis la machine hôte.

Commandes exécutées :

- ps aux (dans le conteneur)
- docker top iso-test (sur la machine hôte)



```
Windows PowerShell x root@8239df8ae874 x + v - □ x
(root@Hamou-laptop)-[~/tp-docker]
# docker run -it --name iso-test ubuntu:22.04 bash
root@8239df8ae874:/# ps aux
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME
COMMAND
root           1  0.2  0.0   4628   3456 pts/0    Ss   11:32   0:00
bash
root           9  0.0  0.0   7064   2688 pts/0    R+   11:32   0:00
ps aux
root@8239df8ae874:/# cat /proc/self/cgroup
0::/
root@8239df8ae874:/# hostname && ip addr
8239df8ae874
bash: ip: command not found
root@8239df8ae874:/# hostname && ip addr show
8239df8ae874
bash: ip: command not found
root@8239df8ae874:/# |
```

second terminal sur la machine hôte

ps aux | grep bash

docker top iso-test

```

Windows x root@82: x root@Ha x + v - □ x
(Message from Kali developers)
This is a minimal installation of Kali Linux, you likely
want to install supplementary tools. Learn how:
⇒ https://www.kali.org/docs/troubleshooting/common-minimum-set
up/
(Run: "touch ~/.hushlogin" to hide this message)
(hamousabs95@Hamou-laptop)~$
$ sudo su
[sudo] password for hamousabs95:
(root@Hamou-laptop)~/home/hamousabs95$
# ps aux | grep bash
hamousa+ 184 0.0 0.0 8412 4992 pts/0 Ss 09:50 0:00
-bash
hamousa+ 227 0.0 0.0 5308 4224 pts/1 Ss+ 09:50 0:00
-bash
root 391 0.0 0.0 5308 4608 pts/4 Ss+ 09:53 0:00
-bash
root 1091 0.0 0.0 9060 5632 pts/5 S 10:05 0:00
bash
root 2270 0.0 0.1 1256472 25728 pts/5 Sl+ 11:32 0:00
docker run -it --name iso-test ubuntu:22.04 bash
hamousa+ 2306 0.2 0.0 8412 4992 pts/6 Ss 11:36 0:00
-bash
root 2347 0.0 0.0 8412 4992 pts/7 S 11:36 0:00
bash
root 2356 33.3 0.0 6608 2048 pts/7 S+ 11:37 0:00
grep --color=auto bash
(root@Hamou-laptop)~/home/hamousabs95$
#

```

```

(root@Hamou-laptop)~/home/hamousabs95$
# docker top iso-test

```

UID	CMD	PID	PPID	C	STIME	TTY	TIME
root	bash	1635	1610	0	11:32	pts/0	00:00:00

Réponse à la Question 8 :

- (a) Combien de processus sont visibles dans chaque cas ? Dans le conteneur, seuls **2 processus** sont visibles (bash et ps), avec les PID 1 et 10 environ. Sur la machine hôte, on voit **tous les processus** du système, y compris le processus bash du conteneur, mais avec un PID complètement différent et élevé.
- (b) Quel mécanisme du noyau explique cette différence ? Cette isolation est due au mécanisme des **Namespaces** du noyau Linux, et plus particulièrement le PID namespace. Il permet au conteneur de ne voir que ses propres processus tout en cachant le reste du système.
- (c) Quelle commande hôte vous a permis de « voir à travers » le namespace PID ? C'est la commande `docker top iso-test`.

3.6 Gestion des ressources avec Cgroups

J'ai appliqué des limites strictes de consommation mémoire et CPU sur un conteneur.

Commandes exécutées :

- `docker run -d --memory="256m" --cpus="0.5" --name cgroup-test ubuntu:22.04 sleep 3600`
- Lecture du fichier `memory.limit_in_bytes` dans l'arborescence `/sys/fs/cgroup/...`

```
(root@Hamou-laptop)-[/home/hamousabs95]
# docker run -d --memory="256m" --cpus="0.5" --name cgroup-test ubuntu:22.04 sleep 3600
ab4a0105d955e28b2096ae01207ae26179502635c77c26e7d64ee94459859a22
```

```
Windows x root@82: x root@Ha x + v - □ x
(root@Hamou-laptop)-[/home/hamousabs95]
# docker inspect cgroup-test | grep -A 5 Memory
    "Memory": 268435456,
    "NanoCpus": 500000000,
    "CgroupParent": "",
    "BlkioWeight": 0,
    "BlkioWeightDevice": [],
    "BlkioDeviceReadBps": [],
--
    "MemoryReservation": 0,
    "MemorySwap": 536870912,
    "MemorySwappiness": null,
    "OomKillDisable": null,
    "PidsLimit": null,
    "Ulimits": [],
    "CpuCount": 0,
    "CpuPercent": 0,

(root@Hamou-laptop)-[/home/hamousabs95]
#
```

Réponse à la Question 9 :

- (a) La valeur `memory.limit_in_bytes` correspond-elle à 256 Mo ? Oui. La valeur affichée est 268435456 octets, ce qui correspond exactement à 256 Mo ($256 \times 1024 \times 1024$).

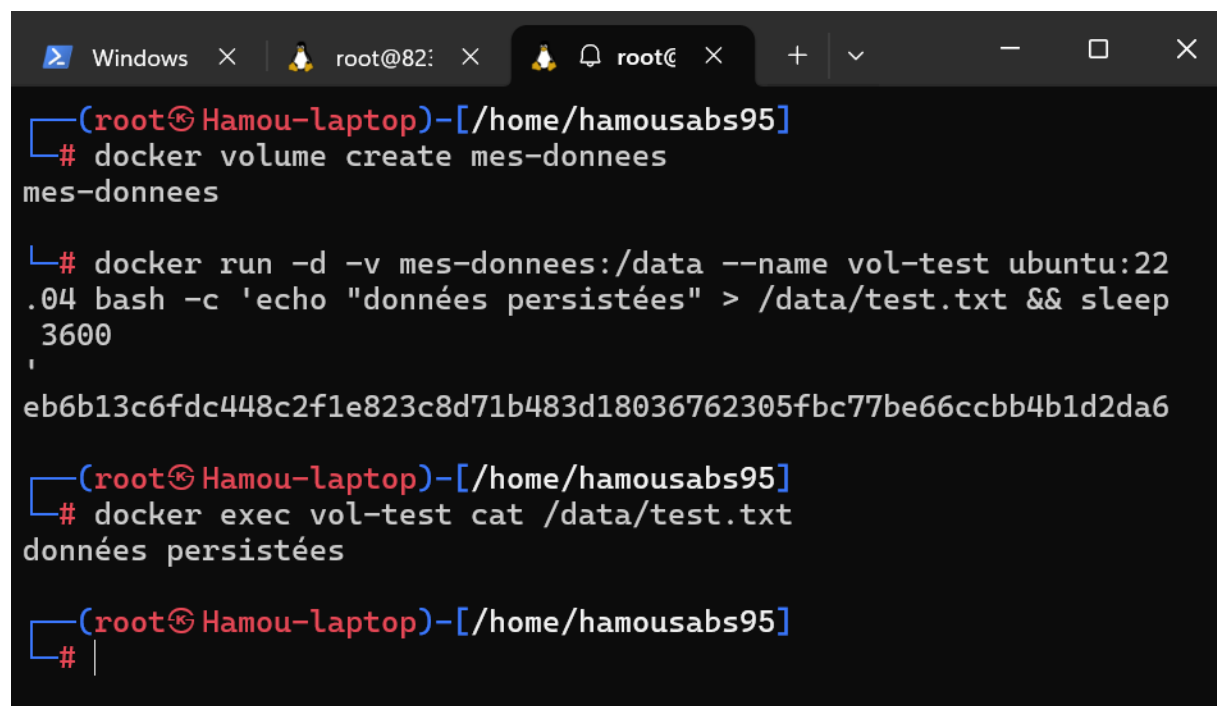
- **(b) Comment les cgroups protègent-ils les autres conteneurs ? Les Cgroups**
(Control Groups) limitent physiquement les ressources qu'un groupe de processus peut consommer. Si le conteneur cgroup-test tente de consommer plus de 256 Mo de RAM, le noyau l'en empêchera ou le tuera (OOM Killer), ce qui protège la machine hôte et les autres conteneurs d'une famine de ressources.
- **(c) Quelle commande Docker permet de modifier les limites d'un conteneur déjà actif ?** La commande est `docker update`.

3.7 Persistance des données avec les volumes

Enfin, j'ai vérifié comment conserver des données même après la suppression d'un conteneur.

Commandes exécutées : Création du volume, écriture du fichier `test.txt`, suppression du conteneur, puis relance d'un nouveau conteneur rattaché au même volume pour lire le fichier.

```
docker volume create mes-donnees docker run -d -v mes-donnees:/data --name vol-test ubuntu:22.04 bash -c 'echo "données persistées" > /data/test.txt && sleep 3600' docker exec vol-test cat /data/test.txt docker stop vol-test && docker rm vol-test docker run --rm -v mes-donnees:/data ubuntu:22.04 cat /data/test.txt
```



```
Windows x root@82: x root@ root@
(root@Hamou-laptop)-[/home/hamousabs95]
# docker volume create mes-donnees
mes-donnees

# docker run -d -v mes-donnees:/data --name vol-test ubuntu:22.04 bash -c 'echo "données persistées" > /data/test.txt && sleep 3600'
eb6b13c6fdc448c2f1e823c8d71b483d18036762305fbc77be66ccbb4b1d2da6

(root@Hamou-laptop)-[/home/hamousabs95]
# docker exec vol-test cat /data/test.txt
données persistées

(root@Hamou-laptop)-[/home/hamousabs95]
#
```

```
Windows x root@82: x root@Ha x + v - □ x
(root@Hamou-laptop)-[/home/hamousabs95]
# docker stop vol-test && docker rm vol-test
vol-test
vol-test

(root@Hamou-laptop)-[/home/hamousabs95]
# ls

(root@Hamou-laptop)-[/home/hamousabs95]
# |
```

```
Windows x root@82: x root@Ha x + v - □ x
(root@Hamou-laptop)-[/home/hamousabs95]
# docker run --rm -v mes-donnees:/data ubuntu:22.04 cat /data/
test.txt
données persistées

(root@Hamou-laptop)-[/home/hamousabs95]
# |
```

Réponse à la Question 10 :

- (a) Le fichier test.txt est-il toujours présent après suppression du premier conteneur ? Pourquoi ? Oui, le fichier est toujours présent. Cela s'explique par le fait qu'un Volume est un objet de stockage géré par Docker de manière totalement indépendante du cycle de vie (création/suppression) du conteneur.
- (b) Quelle est la différence entre un volume Docker géré et un bind mount ? Un volume géré est stocké dans un répertoire contrôlé par Docker (généralement dans /var/lib/docker/volumes/). Un bind mount relie le conteneur à un dossier ou fichier précis et explicite situé n'importe où sur le système de fichiers de la machine hôte.
- (c) Dans quel cas d'usage choisiriez-vous l'un plutôt que l'autre ? Je choisirais un Volume pour persister les données d'une base de données en production (car c'est plus sûr et indépendant de l'OS hôte). Je choisirais un Bind Mount en phase de développement pour lier mon code source, afin que mes modifications sur ma machine soient répercutées instantanément à l'intérieur du conteneur.